

QPU 사용하기

Quantum Processing Unit, 양자 프로세서

2025113574 권나현

2026.05.13

System Software Lab.

주제 선정 이유

- 이론과 실습의 균형
- 컴퓨팅적 함양 기르기
- 양자 컴퓨팅: 양자 역학을 활용하여 가장 강력한 슈퍼컴퓨터조차도 해결할 수 없는 복잡한 문제를 해결하는 기술
- QPU : 양자 컴퓨팅 계산 능력 향상의 핵심
- 따라서 QPU는 양자 컴퓨팅에 필수적인 장치이다.

목차

1. QPU란?
2. Qiskit
3. Building a GHZ state
4. Noise and quantum error
5. Readout error mitigate



1. QPU란?

QPU의 정의

- Quantum Processing Unit, 양자 처리 장치
- 양자 역학의 원리로 큐비트(양자 비트)를 사용해 복잡한 문제를 해결하는 최첨단 처리 하드웨어
- 양자 컴퓨터의 양자 연산 부분을 포함하고 있으며, 양자 컴퓨팅 시스템의 두뇌와 같은 기능.
- 대규모 양자 계산에 최적화됨

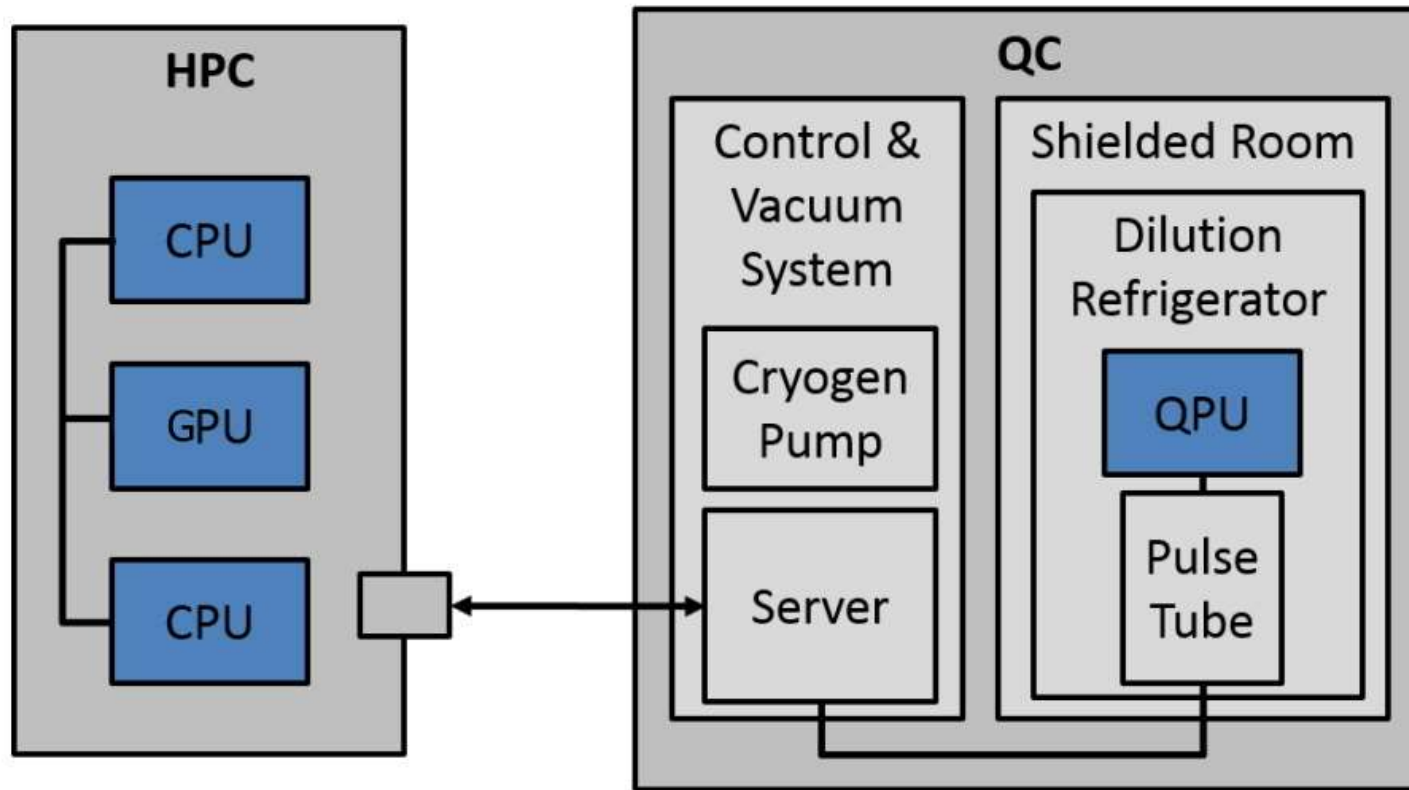
QPU의 효능

- QPU는 가장 강력한 슈퍼컴퓨터보다 양자 알고리즘을 더 잘 처리하도록 설계되었다.
- QPU는 CPU 및 GPU와 함께 고성능 컴퓨팅(HPC) 시스템에 통합되고 있다.
- 사용 사례: 기후 변화, 의약 개발, 인공지능 등의 분야에서 발생하는 문제를 해결하는 데 활용된다.

1. QPU란?

하이브리드-고전 양자 컴퓨터

고전 컴퓨터:
나머지 데이터 처리와
제어

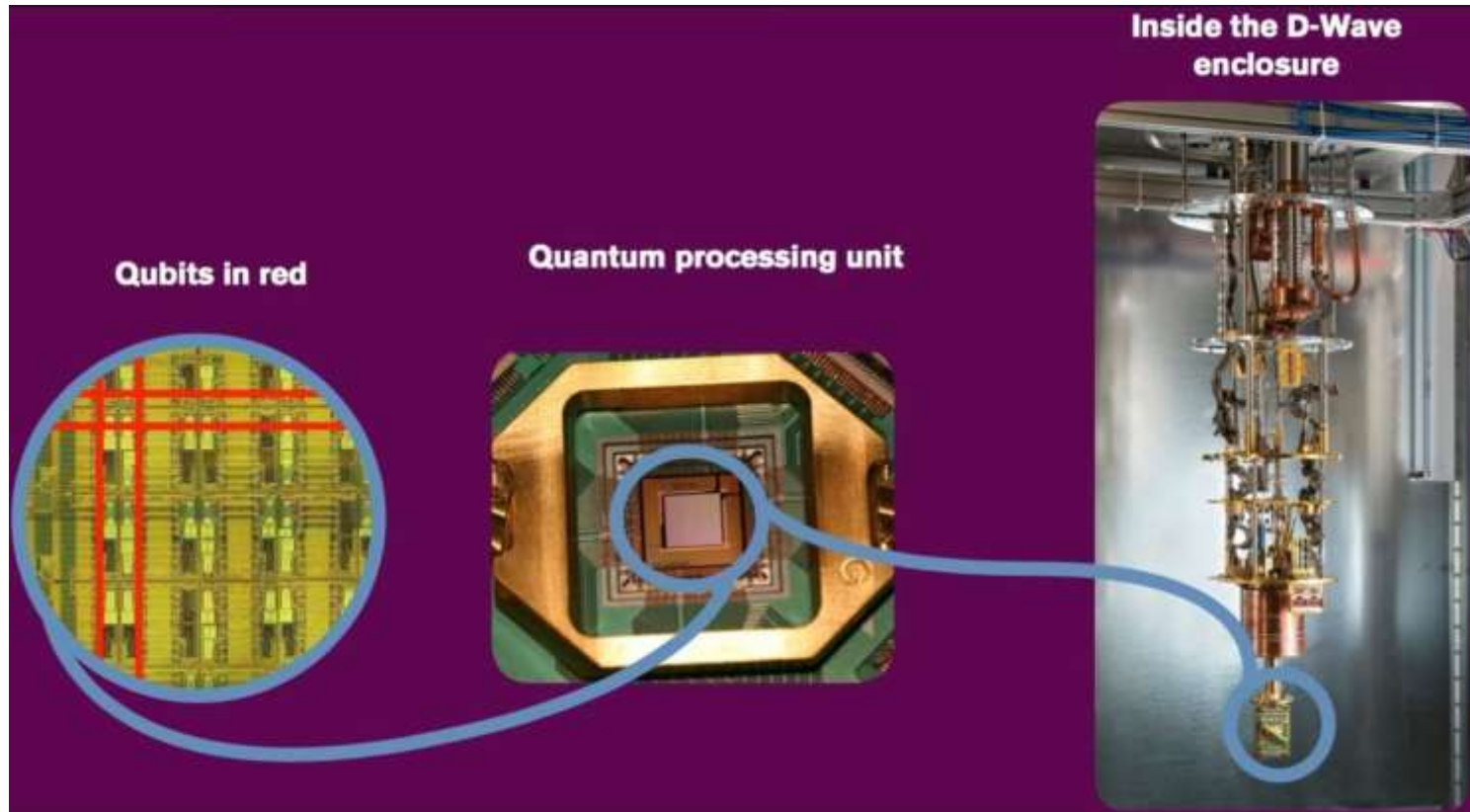


양자 컴퓨터:
수학적으로 복잡한
특정 연산

[이미지: NVIDIA Developer]

1. QPU란? – QPU의 작동 방식

- 큐비트(Qubit, 양자비트)라는 연산 단위가 칩 형태로 배열된 형태
- 외부 환경의 노이즈를 차단하기 위해 극저온(절대영도 근처) 상태에서 작동



1. QPU란? – QPU의 작동 방식

- QPU를 사용하는 코드를 실행하면 일어나는 일

- 
1. 한국 (현위치): 사용자가 노트북에서 작성한 코드가 인터넷을 통해 전송
 2. 미국 (IBM 데이터 센터): 전송된 코드는 실제 양자컴퓨터 장치들이 모여 있는 뉴욕주 근처의 IBM 퀀텀 데이터 센터에 도달
 3. 양자 칩 도달: 코드가 마이크로파(Microwave) 신호로 변환되어 양자컴퓨터의 금속 배선들을 타고 내려가서 맨 밑에 있는 칩(예: ibm_fez, ibm_kingston 등)에 도달
 4. 연산 및 측정: 칩 내부의 초전도 소자들이 양자 중첩과 얽힘 상태를 만들어 계산을 수행하고, 그 결과가 다시 전기 신호로 바뀌어 올라옴

2. Qiskit

- IBM에서 개발한 언어로, 클라우드 시스템을 통해 양자 컴퓨터에 접근하고 회로를 설계하기 위해 만들어진 python 기반의 오픈소스 양자컴퓨팅 프레임워크
- 개인이 Qiskit에서 회로를 설계하여 IBM의 서버로 보내면 IBM은 그것을 실제 양자컴퓨터에서 실험하여 결과를 보내준다.
- 기본적으로 하드웨어 컨트롤, 오류 정정 등 여러 가지 기능들이 있다.



2. Qiskit - IBM Quantum documentation

• 전체적인 구조

Qiskit SDK

로컬 개발 환경

양자 회로를 설계하고 최적화하는 데 필요한 도구 모음

개발자의 로컬 컴퓨터에 설치되어 사용됨

주요 역할: 양자 회로 설계, 트랜스파일, 시각화

(트랜스파일 Transpile: 특정 하드웨어에 맞게 회로 변형)

Qiskit Runtime

실행 환경

설계된 회로를 효율적으로 실행하기 위한 아키텍처

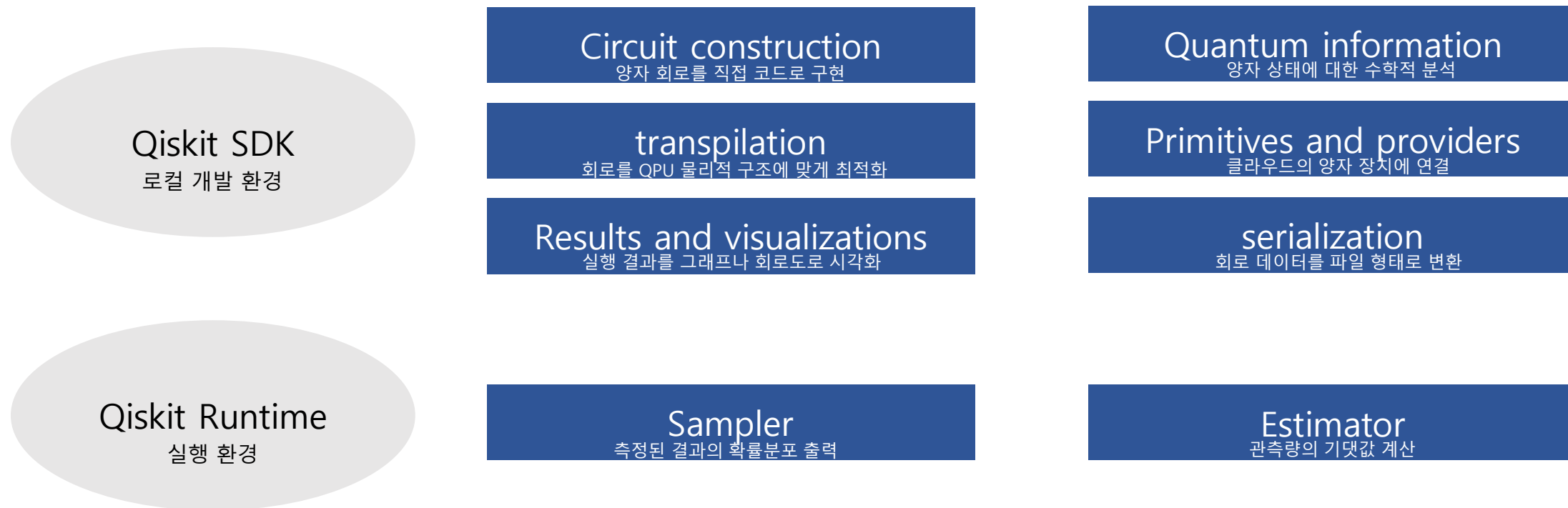
클라우드 기반으로 작동하며 IBM Quantum 하드웨어와 연결됨

주요 역할: 회로 실행 관리, 노이즈 완화(Error Mitigation), 반복 작업 최적화

특징: 회로를 프로그램 단위로 묶어 클라우드에서 실행

2. Qiskit - IBM Quantum documentation

- 세부적 구조



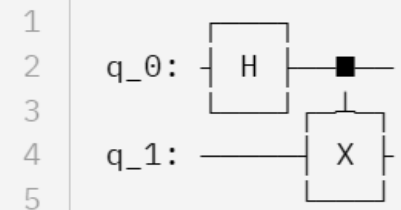
2. Qiskit - IBM Quantum documentation

- 문법

Circuit construction

QuantumCircuit 클래스: 양자 비트(Qubit)를 만들고 게이트(H, CNOT)를 배치하는 도면을 그리는 역할

```
1  from qiskit import QuantumCircuit
2
3  # Create a new circuit with two qubits
4  qc = QuantumCircuit(2)
5
6  # Add a Hadamard gate to qubit 0
7  qc.h(0)
8
9  # Perform a controlled-X gate on qubit 1, controlled by qubit 0
10 qc.cx(0, 1)
11
12 # Return a text drawing of the circuit.
13 qc.draw()
```



2. Qiskit - IBM Quantum documentation

- 문법

Quantum information

Statevector 클래스: 양자 상태를 수학적 벡터 형태로 나타냄. 큐비트 상태 확인에 사용

`qiskit.quantum_info.Statevector(data, dims=None)`

Parameters

- **data** (`np.ndarray` / [list](#) / [Statevector](#) / [Operator](#) / [QuantumCircuit](#) / [circuit.instruction.Instruction](#)) – Data from which the statevector can be constructed. This can be either a complex vector, another statevector, a `Operator` with only one column or a `QuantumCircuit` or `Instruction`. If the data is a circuit or instruction, the statevector is constructed by assuming that all qubits are initialized to the zero state.
- **dims** ([int](#) / [tuple](#) / [list](#) / `None`) – The subsystem dimension of the state (See additional information).

2. Qiskit - IBM Quantum documentation

- 문법

Quantum information

SparsePauliOp 클래스: 여러 개의 파울리 연산자들의 선형 결합을 효율적으로 표현, 계산
일반적으로 관측값을 정의할 때 자주 사용된다.

주요 역할

0이 아닌 계수를 가진 파울리 문자열(예:XYZI)만 저장함으로써 메모리 사용량을 획기적으로 줄임

주요 기능

대수적 연산 지원, 빠른 기댓값 계산(Estimator와 같은 Qiskit Runtime 기능과 함께 사용)

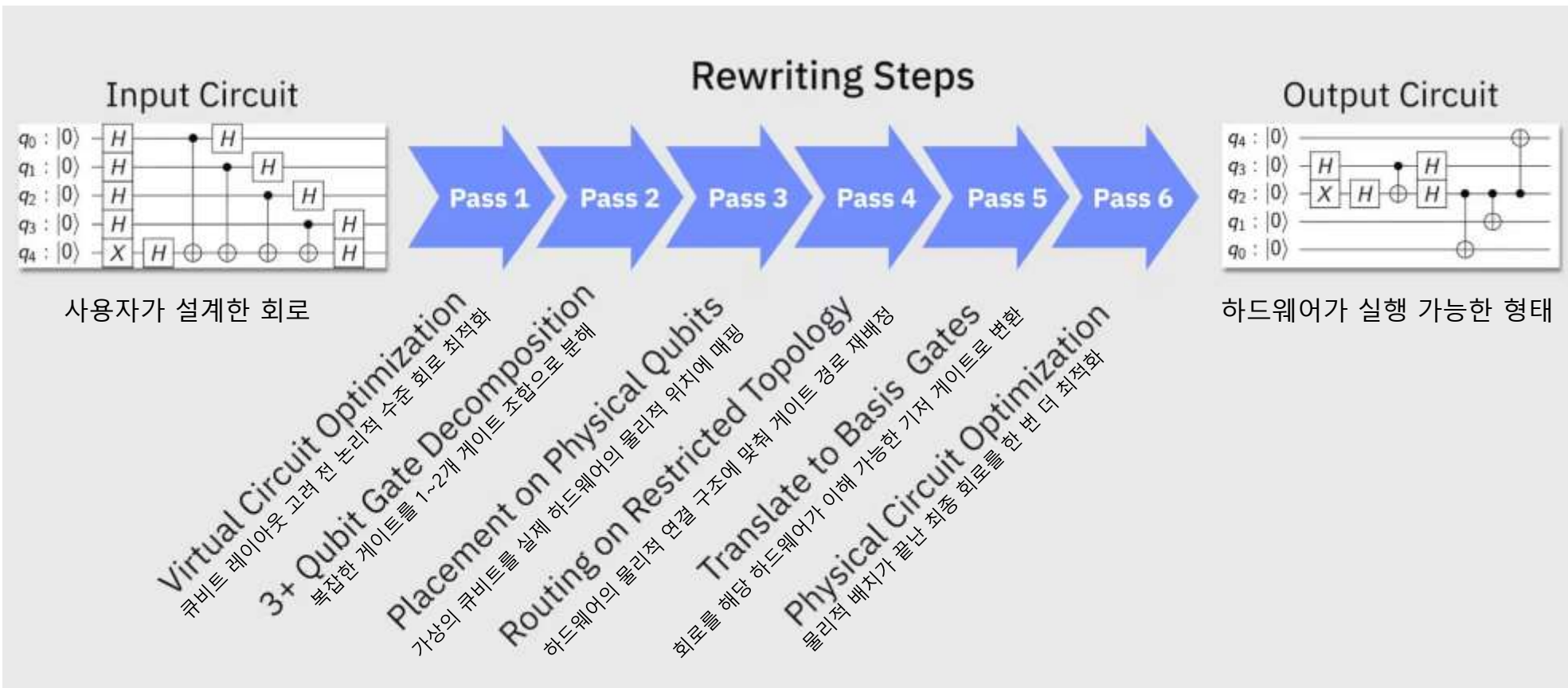
2. Qiskit - IBM Quantum documentation

- 문법

transpilation

회로 최적화 과정

원리)



2. Qiskit - IBM Quantum documentation

• 문법

Qiskit.compiler

transpile 함수: 양자 상태를 수학적인 벡터 형태로 나타냄. 큐비트 상태 확인에 사용

Parameters

- **circuits** (*_CircuitT*) – Circuit(s) to transpile
- **backend** (*Backend* / *None*) – If set, the transpiler will compile the input circuit to this target device. If any other option is explicitly set (e.g., `coupling_map`), it will override the backend's.
- **optimization_level** (*int* / *None*) –

How much optimization to perform on the circuits. Higher levels generate more optimized circuits, at the expense of longer transpilation time.
 - 0: no optimization
 - 1: light optimization
 - 2: heavy optimization
 - 3: even heavier optimization

If `None`, level 2 will be chosen as default.

Returns

The transpiled circuit(s).

Raises

TranspilerError – in case of bad inputs to transpiler (like conflicting parameters) or errors in passes

Return type

_CircuitT

2. Qiskit - IBM Quantum documentation

- 문법

Primitives and providers

apply_layout 함수

주요 역할

가상 큐비트를 실제 하드웨어의 물리적 큐비트로 매핑한 레이아웃 정보를 양자 회로에 직접 적용하는 역할
: 매핑 정보 고정(회로의 가상 큐비트 번호와 실제 칩셋의 물리적 위치), 연산자 정렬(파울리 문자열 순서), 하드웨어 실행 준비

주요 기능

트랜스파일(transpile) 과정의 결과물 후처리, 연산자를 하드웨어 구조에 맞게 변형

2. Qiskit - IBM Quantum documentation

- 문법

Results and visualizations

result() 함수

주요 역할

데이터 추출: 하드웨어나 시뮬레이터에서 연산이 완료된 후, 해당 작업 객체에 저장된 수치 데이터를 가져옴

Parameters

experiment (*str ↗ or QuantumCircuit or int ↗ or None*) – the index of the experiment.

Several types are accepted for convenience:: * str: the name of the experiment. *

QuantumCircuit: the name of the circuit instance will be used. * int: the position of the experiment. * None: if there is only one experiment, returns it.

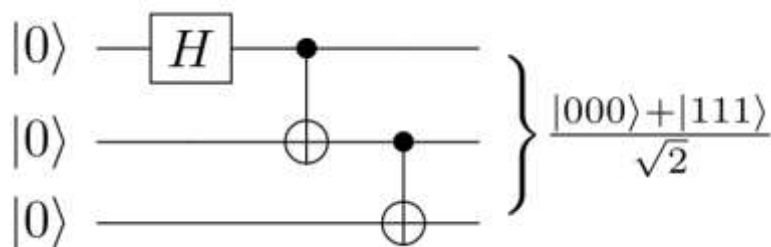
3. Building a GHZ state

• GHZ state란?

: Greenberger-Horne-Zeilinger state

여러 큐비트가 최대치로 얽혀 있는 양자 상태.
모든 큐비트가 얽혀 있으므로 단 하나의 큐비트만 측정해도
나머지 모든 큐비트의 상태가 즉시 결정된다.

$$|GHZ\rangle = \frac{|0\rangle^{\otimes n} + |1\rangle^{\otimes n}}{\sqrt{2}}$$



-3-qubit GHZ state-

<GHZ 상태 설계>

1. H gate : $q[0]$ 중첩 $\left(\begin{matrix} q[0] & q[1] & q[2] & q[3] \\ |0000\rangle \end{matrix} \right)$

$$\frac{1}{\sqrt{2}} (|0000\rangle + |1000\rangle)$$

2. CNOT gate : $q[0] \rightarrow q[1] \rightarrow q[2] \rightarrow q[3]$ 순서
① ② ③

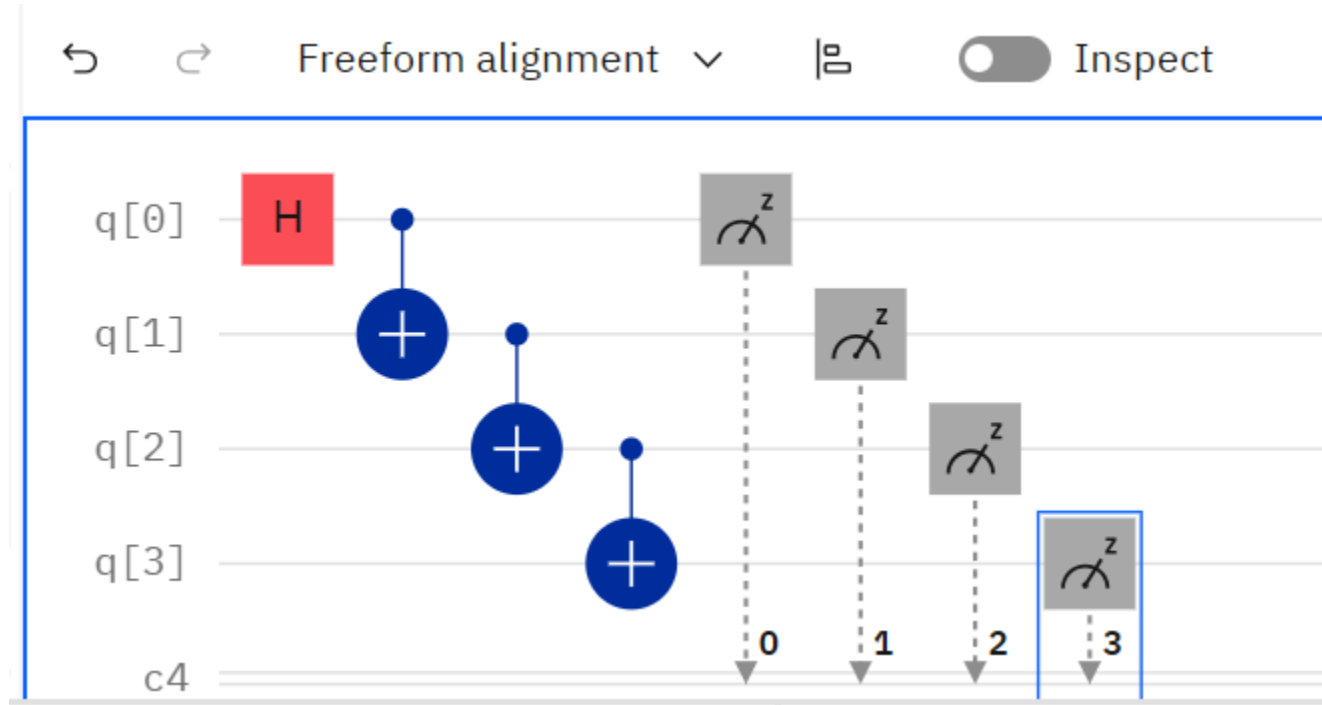
$$\begin{aligned} \textcircled{1} &: \frac{1}{\sqrt{2}} (|0000\rangle + |1100\rangle) \\ \downarrow \\ \textcircled{2} &: \frac{1}{\sqrt{2}} (|0000\rangle + |1110\rangle) \\ \downarrow \\ \textcircled{3} &: \frac{1}{\sqrt{2}} (|0000\rangle + |1111\rangle) \end{aligned}$$

3. $q[3]$ 역시 얽혀 전체 4qubit 측정가능

3. Building a GHZ state

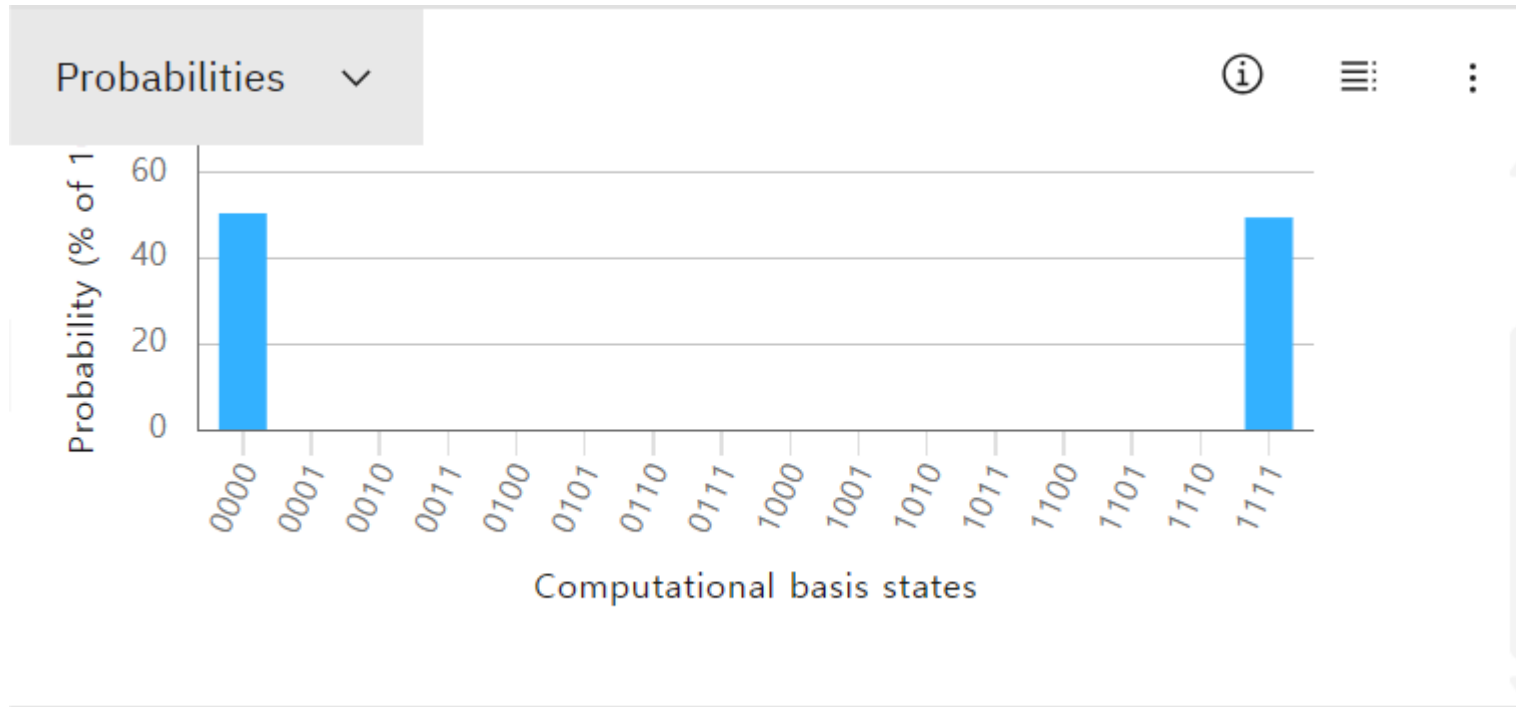
IBM quantum의 composer를 사용하여 GHZ 상태를 만들었다.
(composer: GUI 기반 양자 프로그래밍 도구, 드래그 앤 드롭 방식)

양자 회로



3. Building a GHZ state

예측 상태 결과



3. Building a GHZ state

양자 하드웨어 돌려보기

IBM Quantum Platform

Search

국립금오공과대학교

us-east

[All workloads /](#)
d7tcjfcf3ras73b91jr0

Actions

Details

User

Nahyeon Kwon

Region

Washington DC (us-east)

Instance

ghz-test

Quantum computer

ibm_kingston

Program

executor

of items

1

Tags

Composer

Edit tags

Status and usage

Status

Completed

Total completion time:

4m 38s

Actual QR usage

2s

Created: May 6, 2026, 1:55 PM

Pending: 0s

In progress: May 6, 2026, 1:55 PM
Qiskit Runtime usage: 2s

Completed: May 6, 2026, 2:00 PM

Input options

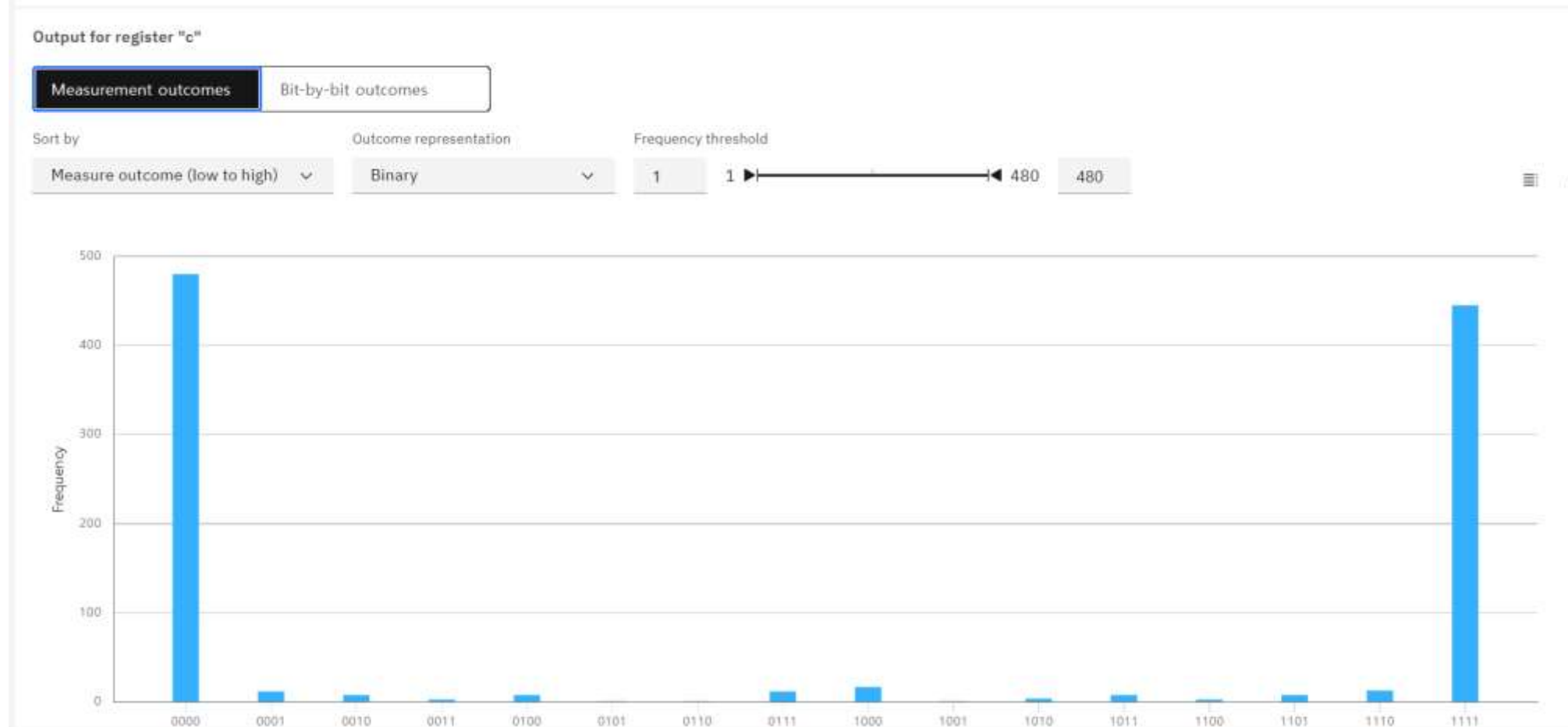
▼

`{init_qubits: true}`

`init_qubits: true`

3. Building a GHZ state

실제 결과 – 노이즈 발생



3. Building a GHZ state

실제 결과

0000> : 505	1011> : 8
0001> : 1	1100> : 1
0011> : 5	1101> : 5
0100> : 1	1110> : 6
0111> : 9	1111> : 468
1000> : 15	

이전의 의도된 상태와 달리 |0000>과 |1111>이 아닌 값들에도 frequency 값이 분포되어 있다.
이를 통해 다양한 원인으로 인한 양자 오류가 발생하여 결과에 영향을 미쳤음을 확인할 수 있다.

4. Noise and quantum error

- 양자 컴퓨터에서는 양자 하드웨어의 물리적인 한계로 인해 노이즈(Noise)가 발생한다.
- 노이즈의 원인: 열적 이완, 큐비트 간 간섭, 제어 신호의 불안정성 등 외부 환경과의 상호작용
- 양자 오류의 종류
 - Decoherence(결맞음 해제): 양자 상태가 유지되는 시간의 한계
 - Gate Error(게이트 오류): 양자 게이트 연산 시 발생하는 불완전성
 - Readout Error(판독 오류): 측정 단계에서 0과 1을 잘못 읽는 오류
 - Crosstalk(크로스토크): 인접한 큐비트 간의 원치 않는 상호작용
- QPU 실행 시 노이즈로 인해 다양한 종류의 양자 오류들이 복합적으로 작용하여 이론적 기댓값을 감소시킨다.

5. Readout error mitigate

- Readout Error(판독 오류): 측정 단계에서 0과 1을 잘못 읽는 오류
- Qiskit api에서 resilience_level을 1로 바꾸는 방법으로 오류를 완화할 수 있다.

resilience_level

Type: UnsetType | int

Default value: Unset

How much resilience to build against errors. Higher levels generate more accurate results, at the expense of longer processing times.

- 0: No mitigation.
- 1: Minimal mitigation costs. Mitigate error associated with readout errors.
- 2: Medium mitigation costs. Typically reduces bias in estimators but is not guaranteed to be zero bias.

Refer to the [Configure error mitigation for Qiskit Runtime](#) guide for more information about the error mitigation methods used at each level.

Default: 1.

5. Readout error mitigate - 실습

```
from qiskit.transpiler.passes.analysis import num_qubits
import numpy as np
from qiskit import QuantumCircuit
from qiskit_ibm_runtime import QiskitRuntimeService, SamplerV2, SamplerOptions
from qiskit.quantum_info import state_fidelity
from qiskit import transpile
from qiskit_ibm_runtime import EstimatorV2 as Estimator
from qiskit.quantum_info import SparsePauliOp
```

```
# IBM 계정 연결(최초 1회)
```

```
QiskitRuntimeService.save_account(
    channel="ibm_quantum_platform",
    token="                (토큰 값)                ",
    instance='            (crn 값)'
)
```

5. Readout error mitigate - 실습

#1. 서비스 연결

```
service = QiskitRuntimeService(channel="ibm_quantum_platform")
real_backend = service.least_busy(simulator=False, operational=True)
print(f"Using backend: {real_backend.name}")
```

#2. 회로 준비

```
nq = 4
qc = QuantumCircuit(nq)
qc.h(0)
qc.cx(0, 1)
qc.cx(0, 2)
qc.cx(0, 3)
```

#3. 실제 장비용 트랜스파일

```
qc_t = transpile(qc, backend=real_backend, optimization_level=3)
num_qubits = qc_t.num_qubits
obs_original = SparsePauliOp("Z" * qc.num_qubits) #관측량
obs_t = obs_original.apply_layout(qc_t.layout)
```

#4. 오류 완화 미적용

```
estimator_raw = Estimator(mode=real_backend, options={"resilience_level" : 0})
print("오류 완화 미적용: 실제 장비에 작업 제출 중...")
job_raw = estimator_raw.run([qc_t, obs_t])
result_raw = job_raw.result()
```

#5. 오류 완화 적용

```
estimator_mitigated = Estimator(mode=real_backend, options={"resilience_level" : 1})
print("오류 완화 적용: 실제 장비에 작업 제출 중...")
job_mitigated = estimator_mitigated.run([qc_t, obs_t])
result_mitigated = job_mitigated.result()
```

#6. 결과 확인

```
expectation_value_raw = result_raw[0].data.evs
expectation_value_mitigated = result_mitigated[0].data.evs
print(f"오류 완화가 적용되지 않은 기댓값( $Z^n$ ): {expectation_value_raw}")
print(f"오류 완화가 적용된 기댓값( $Z^n$ ): {expectation_value_mitigated}")
```

5. Readout error mitigate - 실습

실행 결과

```
... /usr/local/lib/python3.12/dist-packages/samplomatic/__init__.py:20: UserWarning:
You have imported samplomatic==0.18.0 which is in
beta development. Please expect breaking changes between
minor versions and pin your dependencies accordingly.
  _warn_once_per_version(
Using backend: ibm_kingston
ERROR:stevedore.extension:Could not load 'ibm_backend': cannot import name 'ProviderV1' from 'qiskit.providers' (/usr/local/lib/python3.12/dist-packages/qiskit/providers/__init__.py)
ERROR:stevedore.extension:Could not load 'ibm_dynamic_circuits': cannot import name 'ProviderV1' from 'qiskit.providers' (/usr/local/lib/python3.12/dist-packages/qiskit/providers/__init__.py)
ERROR:stevedore.extension:Could not load 'ibm_backend': cannot import name 'ProviderV1' from 'qiskit.providers' (/usr/local/lib/python3.12/dist-packages/qiskit/providers/__init__.py)
ERROR:stevedore.extension:Could not load 'ibm_dynamic_circuits': cannot import name 'ProviderV1' from 'qiskit.providers' (/usr/local/lib/python3.12/dist-packages/qiskit/providers/__init__.py)
오류 완화 미적용: 실제 장비에 작업 제출 중...
오류 완화 적용: 실제 장비에 작업 제출 중...
오류 완화가 적용되지 않은 기댓값(Z^n): 0.8798828125
오류 완화가 적용된 기댓값(Z^n): 0.9866942433643571
```

4큐비트 GHZ 상태에서 관측량의 이론적인 기댓값: 1.0 (파울리 Z 연산에 의해)

오류 완화 미적용 기댓값(약 0.8799) = 약 12% 정도의 오차

오류 완화 적용 기댓값(약 0.9867) = 약 1.3% 정도의 오차

➔ 측정 과정에서 발생한 readout error가 잘 제거되었다.

5. Readout error mitigate

The screenshot displays the IBM Quantum Platform interface for a specific workload. The browser address bar shows the URL: `quantum.cloud.ibm.com/instances/crn%3A%3Av1%3Abluemix%3Apublic%3Aquantum-computing%3Aus-east%3Aa%2F6b9135a7aa304b40a47c670089b5edc4%3Add...`. The page header includes the IBM Quantum Platform logo, a search bar, the user name '국립금오공과대학교', and the region 'us-east'. The main content area shows the workload ID 'd81c0emgbeec73akc6j0' and an 'Actions' button. Below this, there are two main sections: 'Details' and 'Status and usage'. The 'Details' section contains a table with the following information:

User	Region	Program
Nahyeon Kwon	Washington DC (us-east)	estimator
Instance	Mode	# of PUBs
ghz-test	Job	1
Quantum computer		Tags
ibm_fez		

The 'Status and usage' section provides the following information:

- Status: Completed (indicated by a green checkmark)
- Total completion time: 25s
- Actual QR usage: 12s
- Timeline of events:
 - Created: May 12, 2026, 2:52 PM
 - Pending: 1s
 - In progress: May 12, 2026, 2:53 PM (Qiskit Runtime usage: 12s)
 - Completed: May 12, 2026, 2:53 PM

Below these sections is a 'Primitive Unified Blobs (PUBs)' section, which is currently empty. At the bottom, there is a 'Result' section showing a JSON object:

```
{fevs: 1.0737823501044040, ensemble_standard_error: 0.012949227283715592}
```

감사합니다.